

Project Proposal: Finetuning-Friendly Canonicalization of Transformer Checkpoints

Research Plan

May 12, 2026

Abstract

This project studies *function-preserving canonicalization* of pretrained Transformer checkpoints as a way to make downstream adaptation easier. The core observation is simple: many modern Transformer parameterizations contain nontrivial gauge freedom, so multiple checkpoints can implement essentially the same inference function. Practical adaptation algorithms - AdamW, LoRA, QLoRA, DoRA, PiSSA, DPO, PPO/GRPO, gradient clipping, weight decay, and low-precision training - are not invariant to these reparameterizations. Therefore, the representative of a pretrained model inside its gauge orbit matters.

The project goal is to choose a canonical representative that improves SFT/LoRA/QLoRA/RL fine-tuning speed, stability, and final quality without changing initial inference behavior. The strongest first experiments are: (i) exact value-output activation-gradient balancing in grouped-query attention, (ii) exact SwiGLU hidden-channel balancing, and (iii) quantization-aware gauge selection before QLoRA. Given available compute for 0.6B–1B models, the main empirical suite should use Qwen3-0.6B-Base and Llama-3.2-1B as primary small modern bases, plus overtrained or learning-rate-decayed variants as controlled hard-to-adapt checkpoints. Evaluation should use standard 2025-era benchmarks such as AIME, MATH-500, LiveCodeBench, MBPP/MBPP+, BigCodeBench, IFEval, and BFCL.

Contents

1	One-Sentence Thesis	1
2	What Changes Relative to the Previous Direction	1
3	Problem Formulation	2
4	Core Hypotheses	3
5	Exact and Near-Exact Canonicalization Operations	3
5.1	Exact value-output gauge in GQA attention	3
5.2	Value row whitening	3
5.3	Value-output norm balancing	4
5.4	Value-output activation-gradient Fisher balancing	4
5.5	LoRA rank-shaping canonicalization	4
5.6	Exact SwiGLU MLP hidden-channel balancing	5
5.7	RMSNorm gamma folding	5
5.8	Q/K scale canonicalization under QK-Norm	5
5.9	Untied lm-head row-centering	5

5.10	Gauge-LoftQ: quantization-aware canonicalization	6
6	Why This Should Work	6
6.1	AdamW is not gauge invariant	6
6.2	LoRA is especially basis-sensitive	6
6.3	Quantization is not gauge invariant	6
7	Base Model Selection	6
7.1	Operational hardness score	7
7.2	Primary model matrix	7
7.3	Practical ranking	8
7.4	Model families to de-emphasize in the first paper	8
8	Synthetic Hard-to-Finetune Checkpoint Construction	8
8.1	Construction	9
8.2	Hypothesis	9
8.3	Controls	9
9	Task and Benchmark Selection	9
9.1	Primary adaptation tasks	9
9.2	Secondary generalization and non-forgetting checks	10
9.3	Task inclusion rule	10
10	Experimental Matrix	11
10.1	Canonicalizers	11
10.2	Adaptation stacks	11
10.3	Metrics	12
11	Implementation Plan	13
11.1	Phase 0: infrastructure and invariance tests	13
11.2	Phase 1: exact V/O activation-gradient canonicalization	13
11.3	Phase 2: exact MLP hidden-channel balancing	13
11.4	Phase 3: modern-base and hard-checkpoint pilots	14
11.5	Phase 4: main experiments	14
11.6	Phase 5: QLoRA/Gauge-LoftQ	14
12	Main Ablations and Failure Modes	14
12.1	Ablations	14
12.2	Likely failure modes	14
13	Expected Paper Claim	15
13.1	Contributions	15
14	Go/No-Go Criteria	15
14.1	Green-light result	15
14.2	No-go result	16

15 Minimal First Experiment	16
15.1 Stage 0: static invariance and geometry check	16
15.2 Stage 1: cheap main SFT signal	17
15.3 Stage 2: cross-family validation	17
15.4 Stage 3: hard-to-finetune construction	17
15.5 Stage 4: QLoRA/Gauge-LoftQ	17
16 Summary	18

1 One-Sentence Thesis

Transformer checkpoints should be canonicalized before adaptation: choose a function-preserving representative whose local geometry is better matched to the fine-tuning algorithm.

This is not an anti-finetuning or obfuscation project. The objective is constructive: make useful downstream fine-tuning easier while preserving the base model’s initial behavior.

2 What Changes Relative to the Previous Direction

The previous direction treated gauge transformations as a way to sabotage adaptation. That direction is fragile because exact gauge transformations change coordinates, not quotient-level function classes. A strong adaptive user can canonicalize the checkpoint and train in a better-conditioned representative.

The new direction uses the same mathematical object positively. Instead of asking

Can we make adaptation harder?

we ask

Can we make adaptation systematically easier by choosing better coordinates?

The anti-finetuning sections should be removed from the main proposal. The remaining useful pieces are:

1. exact Transformer gauge symmetries;
2. precision-aware functionality preservation;
3. value-output balancing;
4. MLP hidden-channel balancing;
5. LoRA/QLoRA-aware low-rank geometry;
6. RL stability metrics such as KL-per-reward-gain.

3 Problem Formulation

Let f_{θ_0} be a pretrained base Transformer. Let $\mathcal{G}$ be a set of exact or near-exact functionality-preserving transformations. For $g \in \mathcal{G}$, write

$$\theta_g = g \cdot \theta_0.$$

Definition 1 (Precision-aware equivalence set). *Fix an inference precision $\mathbf{prec}$, calibration distribution D_{cal} , distance $d(\cdot, \cdot)$, and tolerance δ . Define*

$$\mathcal{E}_{\delta}^{\mathbf{prec}}(\theta_0) = \{\theta : \mathbb{E}_{x \sim D_{\text{cal}}} d(f_{\theta}(x), f_{\theta_0}(x)) \leq \delta \text{ under precision } \mathbf{prec}\}.$$

Typical distances are logit MSE, token-level KL, perplexity delta, top-k token disagreement, benchmark delta, or generation-level agreement.

Definition 2 (Finetuning-friendly canonicalization). *For a task distribution Π_{ft} and adaptation algorithm class $\mathcal{A}$, choose*

$$g^* = \arg \min_{g \in \mathcal{G}} \mathbb{E}_{\tau \sim \Pi_{\text{ft}}} R_{\tau}(\text{FT}_{\mathcal{A}, b}(g \cdot \theta_0))$$

subject to

$$g \cdot \theta_0 \in \mathcal{E}_{\delta}^{\mathbf{prec}}(\theta_0).$$

The fixed-budget metric is more useful than final-best-score alone:

$$\Delta_{\text{fixed}}(g; \tau, b) = S_{\tau}(\text{FT}_{\mathcal{A}, b}(g \cdot \theta_0)) - S_{\tau}(\text{FT}_{\mathcal{A}, b}(\theta_0)),$$

where S_{τ} is task score. Also measure steps-to-threshold:

$$T_{\alpha}(g; \tau) = \inf\{t : S_{\tau}(\theta_t^g) \geq \alpha\}.$$

A good canonicalizer should reduce T_{α} , improve area under the validation curve, reduce gradient/update outliers, and preserve base capabilities.

4 Core Hypotheses

H1: Gauge choice affects adaptation. AdamW, LoRA, QLoRA, DPO, PPO/GRPO, gradient clipping, and low-bit quantization are coordinate-dependent. Thus two equivalent checkpoints can have measurably different adaptation trajectories.

H2: Activation-gradient balancing is a static natural-gradient proxy. Balancing forward activation covariance and backward gradient covariance inside an exact gauge orbit should improve first-order optimization and low-rank adaptation.

H3: The effect is strongest on hard-to-adapt bases. Canonicalization should matter most for checkpoints whose pretrained parameter geometry is sharp, overtrained, heavily annealed, or poorly matched to downstream adaptation.

H4: Quantized adaptation is especially gauge-sensitive. Quantization is not invariant to gauge choice. A representative whose quantization residual is low-rank-friendly should improve QLoRA convergence and final quality.

5 Exact and Near-Exact Canonicalization Operations

5.1 Exact value-output gauge in GQA attention

Consider a decoder-only Transformer layer with grouped-query attention. For KV group j and each query head h in that group, write

$$v_{t,j} = W_V^{(j)} x_t, \quad z_{t,h} = \sum_{\tau \leq t} a_{t,h,\tau} v_{\tau,j}, \quad y_t = \sum_h W_O^{(h)} z_{t,h}.$$

For each KV group choose $P_j \in \text{GL}(d_h)$ and apply

$$W_V^{(j)} \leftarrow P_j W_V^{(j)}, \quad W_O^{(h)} \leftarrow W_O^{(h)} P_j^{-1} \quad \forall h : \text{grp}(h) = j.$$

Attention weights are unchanged because they depend on Q, K , not V . The head output transforms as $z_{t,h} \leftarrow P_j z_{t,h}$, and the output projection cancels it:

$$W_O^{(h)} P_j^{-1} P_j z_{t,h} = W_O^{(h)} z_{t,h}.$$

Thus this is exact in real arithmetic and should be near-exact under bf16/fp16 if P_j is well-conditioned.

5.2 Value row whitening

For each (ℓ, j) , let $V = W_V^{(\ell,j)}$ and

$$S_V = (VV^\top + \lambda I)^{1/2}.$$

Set

$$P = S_V^{-1}, \quad V' = S_V^{-1} V, \quad O'_h = O_h S_V.$$

Then $V' V'^\top \approx I$. This is a simple baseline, but it can push bad conditioning into O .

5.3 Value-output norm balancing

Let

$$A = VV^\top + \lambda I, \quad B = \sum_{h: \text{grp}(h)=j} O_h^\top O_h + \lambda I.$$

Choose P so that

$$PAP^\top \approx P^{-\top} B P^{-1}.$$

With $X = P^\top P$, solve

$$XAX = B.$$

The SPD solution is

$$X = A^{-1/2} (A^{1/2} B A^{1/2})^{1/2} A^{-1/2}, \quad P = X^{1/2}.$$

This is safer than pure V whitening because it balances the two sides of the exact gauge pair.

5.4 Value-output activation-gradient Fisher balancing

Collect forward channel covariance and backward gradient covariance on a small proxy batch:

$$C = \mathbb{E}[zz^\top] + \lambda I, \quad F = \mathbb{E}[g_z g_z^\top] + \lambda I.$$

The exact gauge action is

$$C' = PCP^\top, \quad F' = P^{-\top}FP^{-1}.$$

Choose P so that $C' = F'$. Equivalently,

$$XCX = F, \quad X = P^\top P.$$

The SPD solution is

$$X = C^{-1/2}(C^{1/2}FC^{1/2})^{1/2}C^{-1/2}, \quad P = X^{1/2}.$$

Interpretation. This is a static K-FAC-like balancing step inside an exact function-preserving orbit. It equalizes forward and backward second moments in the channel basis actually used by LoRA and AdamW.

5.5 LoRA rank-shaping canonicalization

For a weight matrix W and task loss L , the best first-order rank- r update captures the top singular directions of the gradient:

$$G_r(W) = \left(\sum_{i=1}^r \sigma_i^2(\nabla_W L) \right)^{1/2}.$$

Under the value gauge,

$$\nabla_{V'} L = P^{-\top} \nabla_V L.$$

A directly LoRA-friendly objective is

$$P^\star = \arg \max_{P: \text{cond}(P) \leq \kappa_{\max}} \frac{\sum_{i=1}^r \sigma_i^2(P^{-\top} G_V)}{\|P^{-\top} G_V\|_F^2}.$$

This objective should be treated as a more aggressive, task-conditioned canonicalizer. It may overfit to the proxy task distribution, so it should be evaluated after the generic activation-gradient version.

5.6 Exact SwiGLU MLP hidden-channel balancing

For a SwiGLU block

$$\text{MLP}(x) = W_d(\text{silu}(W_g x) \odot W_u x),$$

there is an exact positive hidden-channel scaling symmetry:

$$W_u[i, :] \leftarrow c_i W_u[i, :], \quad W_d[:, i] \leftarrow W_d[:, i] / c_i,$$

with W_g unchanged. This scales the hidden product channel by c_i and then cancels it in W_d .

A norm-balanced choice is

$$c_i = \left(\frac{\|W_d[:, i]\|_2}{\|W_u[i, :]\|_2 + \epsilon} \right)^{1/2}.$$

A data-aware activation-gradient choice uses hidden channel $h_i = \text{silu}(W_g x)_i (W_u x)_i$ and backpropagated gradient $g_{h,i}$:

$$c_i = \left(\frac{\mathbb{E}[g_{h,i}^2] + \epsilon}{\mathbb{E}[h_i^2] + \epsilon} \right)^{1/4}.$$

5.7 RMSNorm gamma folding

For pre-norm layer input

$$\tilde{x} = \text{RMSNorm}(x; \gamma) = \gamma \odot \frac{x}{\text{rms}(x)},$$

fold γ into downstream linear maps:

$$W_Q \leftarrow W_Q \text{diag}(\gamma), \quad W_K \leftarrow W_K \text{diag}(\gamma), \quad W_V \leftarrow W_V \text{diag}(\gamma),$$

with analogous folding into MLP input projections. Then set $\gamma \leftarrow \mathbf{1}$. This is exact for forward evaluation when the norm output is consumed only by those linear maps. It may interact with optimizer weight decay and tied weights, so it should be used as an ablation, not the first main method.

5.8 Q/K scale canonicalization under QK-Norm

Generic Q/K rotations are not safe under RoPE and QK-Norm. However, headwise positive scaling can be near-invariant under RMS/QK normalization:

$$W_Q^{(h)} \leftarrow c_h W_Q^{(h)}, \quad W_K^{(j)} \leftarrow d_j W_K^{(j)}.$$

Canonicalize so that

$$\mathbb{E} \|W_Q^{(h)} x\|_2^2 / d_h \approx 1, \quad \mathbb{E} \|W_K^{(j)} x\|_2^2 / d_h \approx 1.$$

This is not exact because of ϵ terms and learned scales. It belongs in a later experiment after exact V/O and MLP gauges.

5.9 Untied lm-head row-centering

For untied lm head $z = W_U h + b$, softmax is invariant to adding a common scalar to all logits. Therefore

$$W_U \leftarrow W_U - \mathbf{1} a^\top, \quad a = \frac{1}{V} \sum_{v=1}^V W_U[v, :]$$

preserves the output distribution. Do not apply this if input embeddings and lm head are tied.

5.10 Gauge-LoftQ: quantization-aware canonicalization

QLoRA trains LoRA adapters through a frozen 4-bit quantized backbone [21]. LoftQ jointly chooses a quantized matrix and low-rank initialization to reduce the quantization discrepancy [22]. The gauge extension is:

$$\min_{g, Q, \Delta_r} \|g \cdot W - Q(g \cdot W) - \Delta_r\|_F^2, \quad \text{rank}(\Delta_r) \leq r.$$

Equivalently, choose g so the quantization residual has high top- r singular energy and is easy for LoRA to compensate.

6 Why This Should Work

6.1 AdamW is not gauge invariant

Under $V' = PV$, the gradient transforms as

$$G_{V'} = P^{-\top} G_V.$$

A true natural-gradient or full-matrix preconditioner could transform covariantly. AdamW uses coordinate-wise second moments and weight decay in the chosen basis, so

$$\text{AdamW}(P^{-\top} G_V) \neq P \text{AdamW}(G_V)$$

in general. A gauge transformation can therefore act as static preconditioning.

6.2 LoRA is especially basis-sensitive

LoRA restricts updates to

$$\Delta W = BA, \quad \text{rank}(\Delta W) \leq r.$$

The first-order loss decrease available to rank- r updates is controlled by singular value concentration of $\nabla_W L$. Since exact gauges can reshape singular spectra of gradients observed by a module, the same function can be more or less LoRA-friendly.

6.3 Quantization is not gauge invariant

For a nonlinear quantizer Q ,

$$Q(PW) \neq PQ(W)$$

in general. Thus gauge choice before quantization changes the quantized backbone and the low-rank residual that LoRA must learn. This creates a concrete path to improve QLoRA without modifying the adaptation algorithm.

7 Base Model Selection

Given available compute for 0.6B–1B models, Qwen/Llama small bases should be part of the *main* experimental matrix, not merely external validation. The important distinction is:

The literature-backed hard-to-finetune phenomenon comes from checkpoint state, especially overtraining and LR decay; Qwen3-0.6B-Base and Llama-3.2-1B are modern, compute-feasible substrates on which to measure or induce that hardness.

Therefore, the project should use two axes:

1. **Modern architecture axis:** Qwen3-0.6B-Base and Llama-3.2-1B provide realistic dense/GQA Transformer testbeds under feasible compute.
2. **Hardness axis:** overtrained checkpoints, LR-decayed checkpoints, and stable-LR vs decay-to-zero continuation twins provide evidence-backed hard-to-adapt checkpoint states.

Do not claim by default that Qwen or Llama small bases are “notoriously hard to fine-tune.” Instead, claim that they are the correct modern architectures to test, and use an operational hardness score to decide whether a specific model-task pair belongs in the main hard-adaptation suite.

7.1 Operational hardness score

For model m , task τ , budgeted PEFT stack a , and strong reference stack a_{ref} , define

$$H(m, \tau) = 1 - \frac{S_\tau(a(m)) - S_\tau(m)}{S_\tau(a_{\text{ref}}(m)) - S_\tau(m) + \epsilon}.$$

A model-task pair is useful if:

1. the base model has not saturated the benchmark;
2. the reference adaptation improves it;
3. the cheap/budgeted adaptation struggles, is unstable, or needs many steps;
4. the task is not impossible or purely data-contamination-driven.

High H means the model is hard for the budgeted fine-tuning stack but still learnable by a stronger reference.

7.2 Primary model matrix

Candidate	Why include it	Use in this project
Qwen3-0.6B-Base	Modern 2025 dense Qwen-family base model. The model card lists 0.6B parameters, 28 layers, GQA with 16 query heads and 8 KV heads, and 32k context length [6].	First primary model. It is cheap enough for multi-seed LoRA/QLoRA sweeps and directly supports exact V/O GQA canonicalization. Use native checkpoint first, then optionally construct continuation twins.
Llama-3.2-1B base	Llama-family 1.23B text-only base with GQA and shared embeddings; the model card lists 1B/3B pretrained and instruction-tuned variants and 128k context for text-only models [7].	Second primary model. It tests whether effects transfer outside Qwen while staying within a feasible compute envelope. It is also important because many fine-tuning stacks target Llama-style architectures.
OLMo-1B over-trained/intermediate checkpoints	Springer et al. report “catastrophic overtraining”: extended pretraining can improve base performance but hurt final performance after fine-tuning; they specifically compare OLMo-1B token budgets such as 2.3T vs 3T [1]. OLMo releases code and checkpoints for controlled research [4].	Literature-backed hardness anchor. Use to avoid the criticism that observed hardness is merely a Qwen/Llama idiosyncrasy or training-loop artifact.

Custom Qwen/Llama continuation twins	Yano et al. find that pretraining without LR decay improves downstream SFT and that decay-based schedules can lead to sharper minima and worse adaptability [2].	Strongest causal experiment if compute allows. Continue-pretrain the same base under matched data/tokens: stable LR vs decay-to-zero. Then test whether canonicalization recovers adaptation speed.
--	--	--

7.3 Practical ranking

1. **First smoke test: Qwen3-0.6B-Base.** It is the cleanest starting point because it is small, modern, GQA-based, and cheap enough for many ablations.
2. **First transfer test: Llama-3.2-1B.** Run the same canonicalizers and LoRA configs. If Qwen improves but Llama does not, the method may be architecture- or implementation-specific.
3. **First hardness anchor: OLMo-1B overtrained/intermediate.** Use this to connect the project to the hard-to-finetune literature.
4. **First causal hardness construction: Qwen3-0.6B continuation twins.** Stable-LR vs decay-to-zero continuation is the most direct way to test your hypothesis about LR decay creating low-plasticity checkpoints.

7.4 Model families to de-emphasize in the first paper

- **Llama 4 Scout/Maverick:** useful later, but MoE/multimodal details and size make it a confounded first target.
- **Very large Qwen/Llama variants:** unnecessary until the 0.6B/1B regime shows a robust signal.
- **Models without clean base checkpoints:** less useful because instruction tuning/RLHF can obscure whether canonicalization is helping adaptation or fighting previous post-training.

8 Synthetic Hard-to-Finetune Checkpoint Construction

The cleanest experiment is to manufacture hard-to-adapt bases under controlled training.

8.1 Construction

Start from Qwen3-0.6B-Base or Llama-3.2-1B, or from an OLMo-style checkpoint with accessible training artifacts, and continue pretraining under matched data and token budget:

1. **Stable-LR checkpoint:** warmup-stable-only or constant-LR continuation.
2. **Decay-to-zero checkpoint:** same data and compute, but cosine or linear decay to zero.
3. **Overtrained checkpoint:** repeated or extended token budget after the stable model has reached strong pretraining loss.

For Qwen/Llama, the continuation budget does not need to be large enough to produce a new strong base model. The goal is to create matched checkpoint states with different plasticity while holding architecture, tokenizer, and data fixed. Qwen3-0.6B-Base is the most convenient first target because its GQA geometry is small enough for exhaustive canonicalizer ablations.

The key comparison is not just base perplexity. The key comparison is downstream adaptivity:

lower pretraining loss may coexist with worse post-SFT performance.

8.2 Hypothesis

Decay-to-zero or overtraining moves the model into a sharper, more sensitive, less adapter-friendly representative or basin. Canonicalization cannot change the function-level basin, but it may improve the coordinate geometry seen by finite-budget adaptation.

8.3 Controls

1. Match tokenizer, architecture, data, and total tokens.
2. Evaluate base perplexity and base benchmark scores before adaptation.
3. Run full LR sweeps for LoRA/QLoRA so canonicalization is not merely changing the optimal learning rate.
4. Include random gauges with the same condition-number constraints as negative controls.
5. Include full SFT or high-rank LoRA as an upper reference.

9 Task and Benchmark Selection

The task suite should be standard, diverse, and still relevant in 2025+. It should avoid tasks where ordinary SFT is known to be nearly hopeless or where the base model has already saturated the benchmark.

9.1 Primary adaptation tasks

Domain	Train data	Evaluation	Role
Math reasoning	OpenMathReasoning, NuminaMath-CoT, or a decontaminated subset. OpenMathReasoning is a 2025 large-scale math reasoning dataset with hundreds of thousands to millions of math solutions depending on version [11].	AIME 2024/2025/2026 when available, MATH-500, possibly Omni-MATH. MATH is a 12.5K competition-math dataset and MATH-500 is a common 500-problem evaluation subset [10].	Primary

Code generation	MBPP train/sanitized train, CodeFeedback-Filtered-Instruction, Magicoder-style code instruction data.	MBPP/MBPP+, LiveCodeBench, BigCodeBench. MBPP has 974 entry-level Python tasks [12]; LiveCodeBench continuously collects fresh contest problems to reduce contamination [13]; BigCodeBench targets more realistic and complex function-call-heavy code tasks [14].	Primary
Instruction following	Tulu-3 SFT mixture or a small high-quality instruction mixture. Tulu 3 provides open post-training data, code, and recipes [16].	IFEval, AlpacaEval 2, internal format-following tests. IFEval uses verifiable natural-language constraints and around 500 prompts [15].	Primary
Tool/function calling	BFCL/OpenFunctions-style data or tool-call instruction data.	BFCL V4 or current BFCL. BFCL evaluates tool/function calling across real-world function-use settings and is periodically updated [17].	Primary/optional

9.2 Secondary generalization and non-forgetting checks

- **MMLU-Pro:** use as a robustness/general-knowledge reasoning check, not necessarily as the main SFT-improvement target. MMLU-Pro is designed to be more challenging and reasoning-intensive than MMLU, with 10 answer options and over 12K questions across 14 domains [18].
- **GPQA-Diamond:** use as a hard scientific reasoning check. GPQA contains expert-written graduate-level biology, physics, and chemistry questions [19]. It may be too hard for ordinary SFT to improve reliably, so treat it as secondary.
- **LiveBench:** use as a contamination-aware aggregate check, because it is updated and scores with objective ground truth [20].

9.3 Task inclusion rule

For every candidate task τ , first run a pilot:

$$S_{\text{base}}, \quad S_{\text{LoRA}}, \quad S_{\text{full/high-rank}}.$$

Include τ in the main suite only if

$$S_{\text{full/high-rank}} - S_{\text{base}} \geq \Delta_{\text{min}}$$

and the benchmark is not saturated. If both LoRA and full/reference SFT fail to improve it, use it only as a non-forgetting metric.

10 Experimental Matrix

10.1 Canonicalizers

1. Original checkpoint.
2. Random exact gauge with matched condition-number clipping.
3. V row whitening.
4. V/O norm balancing.
5. V/O activation-gradient balancing.
6. MLP norm balancing.
7. MLP activation-gradient balancing.
8. Combined exact canonicalization: $V/O + \text{MLP}$.
9. Gauge-LoftQ for QLoRA.
10. Optional near-exact Q/K scale canonicalization.

10.2 Adaptation stacks

1. LoRA on W_V only.
2. LoRA on W_V, W_O .
3. LoRA on attention projections.
4. All-linear LoRA.
5. QLoRA all-linear.
6. PiSSA/DoRA baselines.
7. Full SFT or high-rank LoRA as upper reference.

PiSSA and DoRA are useful because they are stronger PEFT baselines: PiSSA changes LoRA initialization using principal singular components [23], while DoRA decomposes pretrained weights into magnitude and direction and applies LoRA to directional updates [24].

10.3 Metrics

For SFT/LoRA/QLoRA:

- validation loss vs steps;
- benchmark score vs steps;
- area under learning curve;
- steps to threshold;

- best score under fixed compute;
- gradient norm variance;
- update norm distribution;
- LoRA factor norm growth;
- top- r singular energy of module gradients;
- base benchmark preservation.

For RL/DPO/PPO/GRPO:

- reward vs updates;
- KL to reference vs updates;
- KL-per-reward-gain;
- clip fraction;
- entropy;
- instability/divergence rate;
- final score under equal sample budget.

For QLoRA:

- quantization reconstruction error;
- residual singular spectrum;
- fraction of quantization residual captured by rank r ;
- downstream score vs steps;
- post-merge quality if merging adapters.

11 Implementation Plan

11.1 Phase 0: infrastructure and invariance tests

1. Implement checkpoint rewrite utilities for V/O and MLP hidden scaling.
2. Verify fp32 exact invariance by comparing logits before and after transformation.
3. Verify bf16/fp16 drift on calibration prompts.
4. Add singular-value clipping:

$$\sigma(P) \in [\sigma_{\min}, \sigma_{\max}].$$
5. Save canonicalization metadata for reversibility and ablation.

11.2 Phase 1: exact V/O activation-gradient canonicalization

For each layer ℓ and KV group j :

1. Run proxy batches through the model.
2. Hook value-path channel activations $z_{\ell,j}$.
3. Backpropagate proxy loss and hook gradients $g_{z_{\ell,j}}$.
4. Estimate

$$C_{\ell,j} = \mathbb{E}[zz^\top] + \lambda I, \quad F_{\ell,j} = \mathbb{E}[g_z g_z^\top] + \lambda I.$$

5. Solve

$$X_{\ell,j} C_{\ell,j} X_{\ell,j} = F_{\ell,j}, \quad P_{\ell,j} = X_{\ell,j}^{1/2}.$$

6. Clip $P_{\ell,j}$, apply $W_V \leftarrow P W_V$ and $W_O \leftarrow W_O P^{-1}$.

11.3 Phase 2: exact MLP hidden-channel balancing

For each SwiGLU block:

1. Hook hidden channel $h_i = \text{silu}(W_g x)_i (W_u x)_i$.
2. Hook gradient $g_{h,i}$.
3. Estimate

$$c_i = \left(\frac{\mathbb{E}[g_{h,i}^2] + \epsilon}{\mathbb{E}[h_i^2] + \epsilon} \right)^{1/4}.$$

4. Clip $c_i \in [c_{\min}, c_{\max}]$.
5. Apply $W_u[i, :] \leftarrow c_i W_u[i, :]$ and $W_d[:, i] \leftarrow W_d[:, i] / c_i$.

11.4 Phase 3: modern-base and hard-checkpoint pilots

1. Start with Qwen3-0.6B-Base and Llama-3.2-1B because they fit the available compute budget.
2. Run constrained PEFT pilots: rank $r \in \{4, 8, 16, 32\}$, module subsets, QLoRA vs bf16 LoRA, and small data budgets.
3. Compute hardness score $H(m, \tau)$ for math/code/instruction tasks.
4. For the most promising family, construct continuation twins: stable-LR vs decay-to-zero, matched data, matched tokens.
5. Add OLMo overtraining checkpoints if available, because they provide the strongest literature-backed hard-adaptation evidence.
6. Select 2–3 model-task pairs with high hardness and reference learnability.

11.5 Phase 4: main experiments

Run the full canonicalizer matrix on selected hard model-task pairs. Use at least 3 seeds and a small LR grid per method. If canonicalization only shifts the optimal learning rate, it is not enough for a strong claim.

11.6 Phase 5: QLoRA/Gauge-LoftQ

Measure whether gauge choice before 4-bit quantization makes the quantization residual more rank- r recoverable and improves QLoRA. Compare against vanilla QLoRA and LoftQ.

12 Main Ablations and Failure Modes

12.1 Ablations

1. **Random gauge control:** same condition number as the proposed gauge.
2. **Weight-only vs data-aware:** compare norm balancing to activation-gradient balancing.
3. **Forward-only vs backward-aware:** compare C whitening, F whitening, and C/F balancing.
4. **Module locality:** attention-only, MLP-only, combined.
5. **Optimizer dependence:** AdamW vs stronger matrix-aware optimizers if feasible.
6. **LR sweep:** ensure gains persist under tuned learning rates.
7. **Rank sweep:** LoRA ranks $r \in \{4, 8, 16, 32, 64\}$.

12.2 Likely failure modes

Learning-rate confounding. Canonicalization may simply rescale gradients and shift the best LR. Mitigation: LR sweeps and learning-rate-normalized comparisons.

Proxy-task overfitting. Rank-shaping can overfit to the proxy gradients. Mitigation: keep activation-gradient balancing as the main generic method and evaluate cross-task transfer.

Numerical drift. Large P can be exact in fp32 but unstable in bf16/fp16/int4. Mitigation: condition-number clipping and drift thresholds.

No effect under strong optimizers. If Shampoo/SOAP/Muon-style optimizers remove the gain, that is still informative: the method is a cheap static substitute for expensive preconditioning.

Task non-learnability. Some benchmarks are common but not good SFT targets. Mitigation: pilot inclusion rule and use hard benchmarks as secondary non-forgetting checks.

13 Expected Paper Claim

A clean paper claim:

Function-preserving canonicalization of Transformer checkpoints improves finite-budget adaptation because practical fine-tuning algorithms are coordinate-dependent.

Possible title:

CanonFT: Finetuning-Friendly Canonicalization of Transformer Checkpoints

Alternative title:

GaugeFix: Choosing Adapter-Friendly Representatives of Pretrained Transformers

13.1 Contributions

1. Formalize finetuning-friendly canonicalization as static preconditioning over a functionality-preserving gauge orbit.
2. Derive exact value-output activation-gradient balancing for GQA attention.
3. Derive exact SwiGLU hidden-channel activation-gradient balancing.
4. Introduce gauge-aware QLoRA/LoftQ as low-rank-friendly quantization residual shaping.
5. Evaluate on literature-backed and synthetically constructed hard-to-adapt base checkpoints.
6. Show improvements in sample efficiency, stability, and fixed-budget performance across math, code, instruction following, and possibly function calling.

14 Go/No-Go Criteria

14.1 Green-light result

Proceed to paper if at least two exact canonicalizers satisfy:

1. negligible initial drift;
2. robust improvement after LR sweep;
3. improvement on at least two domains or two model families;
4. interpretable geometry change, e.g. better gradient rank concentration or lower update variance;
5. no major degradation on secondary non-forgetting benchmarks.

A strong target is:

5–20% fewer steps to threshold

or

1–3 absolute points fixed-budget improvement

on at least one meaningful benchmark, after tuning baselines fairly.

14.2 No-go result

Do not pursue the paper if:

1. all gains disappear under LR sweeps;
2. gains occur only for one fragile module choice;
3. bf16/fp16 drift is too large;
4. random gauges perform as well as proposed canonicalizers;
5. the only improvements are on benchmarks where the base model was already saturated or where reference SFT also fails.

15 Minimal First Experiment

The minimal experiment should now use the feasible 0.6B–1B models directly, rather than treating them as late external-validation models.

15.1 Stage 0: static invariance and geometry check

1. Model: Qwen3-0.6B-Base first; then Llama-3.2-1B. Add Qwen2.5-0.5B/1.5B only if a pure-pretraining Qwen control is needed.
2. Canonicalizers: original, random gauge with matched condition number, V/O norm balancing, V/O activation-gradient balancing, MLP activation-gradient balancing, combined.
3. Safety metrics: fp32 logit MSE, bf16 token-level KL, perplexity delta, top- k token disagreement, and generation-level sanity prompts.
4. Geometry metrics: $\text{cond}(C)$, $\text{cond}(F)$, $\text{cond}(C^{1/2}FC^{1/2})$, top- r gradient energy, LoRA factor norm growth, and update-norm variance.

15.2 Stage 1: cheap main SFT signal

1. Model: Qwen3-0.6B-Base.
2. Task: math SFT on a decontaminated OpenMathReasoning/NuminaMath subset; evaluate MATH-500 and AIME 2024/2025.
3. Fine-tuning: all-linear LoRA with ranks 8, 32, and 64; small LR sweep; 3 seeds.
4. Metrics: validation loss curve, MATH-500, AIME, steps-to-threshold, AUC-over-budget, top- r gradient energy, bf16 drift.

This should be the first go/no-go. If V/O activation-gradient balancing and/or MLP balancing does not produce any robust learning-curve improvement after LR sweep, the project is weak as a generic SFT preconditioning story.

15.3 Stage 2: cross-family validation

Repeat the strongest Stage-1 setting on:

1. Llama-3.2-1B, as the Llama-style GQA check;
2. optionally Qwen2.5-0.5B/1.5B base, as a pure-pretraining Qwen control;
3. optionally Qwen3-1.7B-Base, as the same-family scale-up.

Do not require all models to improve. A realistic target is improvement on Qwen3 plus one cross-family model, with random-gauge controls failing.

15.4 Stage 3: hard-to-finetune construction

Construct a matched pair by continuing pretraining one model, preferably Qwen3-0.6B-Base:

1. stable-LR continuation;
2. cosine or linear decay-to-zero continuation;
3. optionally overtrained continuation beyond the normal stopping point.

Then compare adaptation under original versus canonicalized representatives. This is the cleanest test of the central hypothesis:

canonicalization helps most when the base is less plastic under ordinary fine-tuning.

15.5 Stage 4: QLoRA/Gauge-LoftQ

If SFT signal is present, run QLoRA with NF4 or comparable 4-bit quantization. Compare ordinary QLoRA, LoftQ-style initialization, and Gauge-LoftQ. The key metric is not only quantization error but whether the residual is low-rank-adaptor-friendly:

$$\frac{\sum_{i \leq r} \sigma_i^2(W - Q(W))}{\|W - Q(W)\|_F^2}.$$

If Stage 1 and Stage 4 both fail, the project is likely not worth scaling.

16 Summary

- The project should be purely about canonicalization for easier adaptation, not anti-finetuning.
- The main theoretical object is a functionality-preserving gauge orbit.
- The main practical claim is static preconditioning for coordinate-dependent optimizers and adaptors.
- The first method should be exact V/O activation-gradient balancing.
- The second method should be exact SwiGLU hidden-channel balancing.
- The third method should be gauge-aware QLoRA/LoftQ.

- Given available compute, Qwen3-0.6B-Base and Llama-3.2-1B should be main experimental models rather than merely external-validity models.
- The most defensible hard-to-finetune evidence still comes from overtrained or LR-decayed checkpoints, so the cleanest plan is to induce hardness via continuation twins on small modern bases.
- The main tasks should be math, code, instruction following, and optionally function calling; GPQA/MMLU-Pro/LiveBench are better as secondary robustness checks unless pilot SFT shows clear learnability.

References

- [1] Jacob Mitchell Springer, Sachin Goyal, Kaiyue Wen, Tanishq Kumar, Xiang Yue, Sadhika Malladi, Graham Neubig, and Aditi Raghunathan. Overtrained Language Models Are Harder to Fine-Tune. arXiv:2503.19206, 2025. <https://arxiv.org/abs/2503.19206>
- [2] Kazuki Yano, Shun Kiyono, Sosuke Kobayashi, Sho Takase, and Jun Suzuki. Pre-training LLM without Learning Rate Decay Enhances Supervised Fine-Tuning. arXiv:2603.16127, 2026. <https://arxiv.org/abs/2603.16127>
- [3] Mark Rofin, Aditya Varre, and Nicolas Flammarion. (How) Learning Rates Regulate Catastrophic Overtraining. arXiv:2604.13627, 2026. <https://arxiv.org/abs/2604.13627>
- [4] Dirk Groeneveld et al. OLMo: Accelerating the Science of Language Models. 2024. <https://allenai.org/blog/olmo-open-language-model-87ccfc95f580>
- [5] An Yang et al. Qwen3 Technical Report. arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>
- [6] Qwen Team. Qwen3-0.6B-Base model card. Hugging Face, 2025. <https://huggingface.co/Qwen/Qwen3-0.6B-Base>
- [7] Meta. Llama-3.2-1B model card. Hugging Face, 2024. <https://huggingface.co/meta-llama/Llama-3.2-1B>
- [8] Google and Hugging Face Transformers. Gemma 3 model documentation. 2025. https://huggingface.co/docs/transformers/model_doc/gemma3
- [9] Hugging Face. Welcome Llama 4 Maverick and Scout on Hugging Face. 2025. <https://huggingface.co/blog/llama4-release>
- [10] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS Datasets and Benchmarks, 2021. <https://arxiv.org/abs/2103.03874>
- [11] Igor Moshkov et al. AIMO-2 Winning Solution: Building State-of-the-Art Mathematical Reasoning Models with OpenMathReasoning Dataset. arXiv:2504.16891, 2025. <https://arxiv.org/abs/2504.16891>
- [12] Jacob Austin et al. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021. <https://arxiv.org/abs/2108.07732>

- [13] Naman Jain et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974, 2024. <https://arxiv.org/abs/2403.07974>
- [14] Terry Yue Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. arXiv:2406.15877, 2024. <https://arxiv.org/abs/2406.15877>
- [15] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-Following Evaluation for Large Language Models. arXiv:2311.07911, 2023. <https://arxiv.org/abs/2311.07911>
- [16] Nathan Lambert et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124, 2024. <https://arxiv.org/abs/2411.15124>
- [17] Shishir G. Patil et al. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. ICML, 2025. <https://proceedings.mlr.press/v267/patil25a.html>
- [18] Yubo Wang et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. NeurIPS Datasets and Benchmarks, 2024. <https://arxiv.org/abs/2406.01574>
- [19] David Rein et al. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. arXiv:2311.12022, 2023. <https://arxiv.org/abs/2311.12022>
- [20] Colin White et al. LiveBench: A Challenging, Contamination-Limited LLM Benchmark. arXiv:2406.19314, 2024. <https://arxiv.org/abs/2406.19314>
- [21] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314, 2023. <https://arxiv.org/abs/2305.14314>
- [22] Yuhang Li et al. LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models. arXiv:2310.08659, 2023. <https://arxiv.org/abs/2310.08659>
- [23] Fanxu Meng, Zhaohui Wang, and Muhan Zhang. PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models. arXiv:2404.02948, 2024. <https://arxiv.org/abs/2404.02948>
- [24] Shih-Yang Liu et al. DoRA: Weight-Decomposed Low-Rank Adaptation. ICML, 2024. <https://arxiv.org/abs/2402.09353>